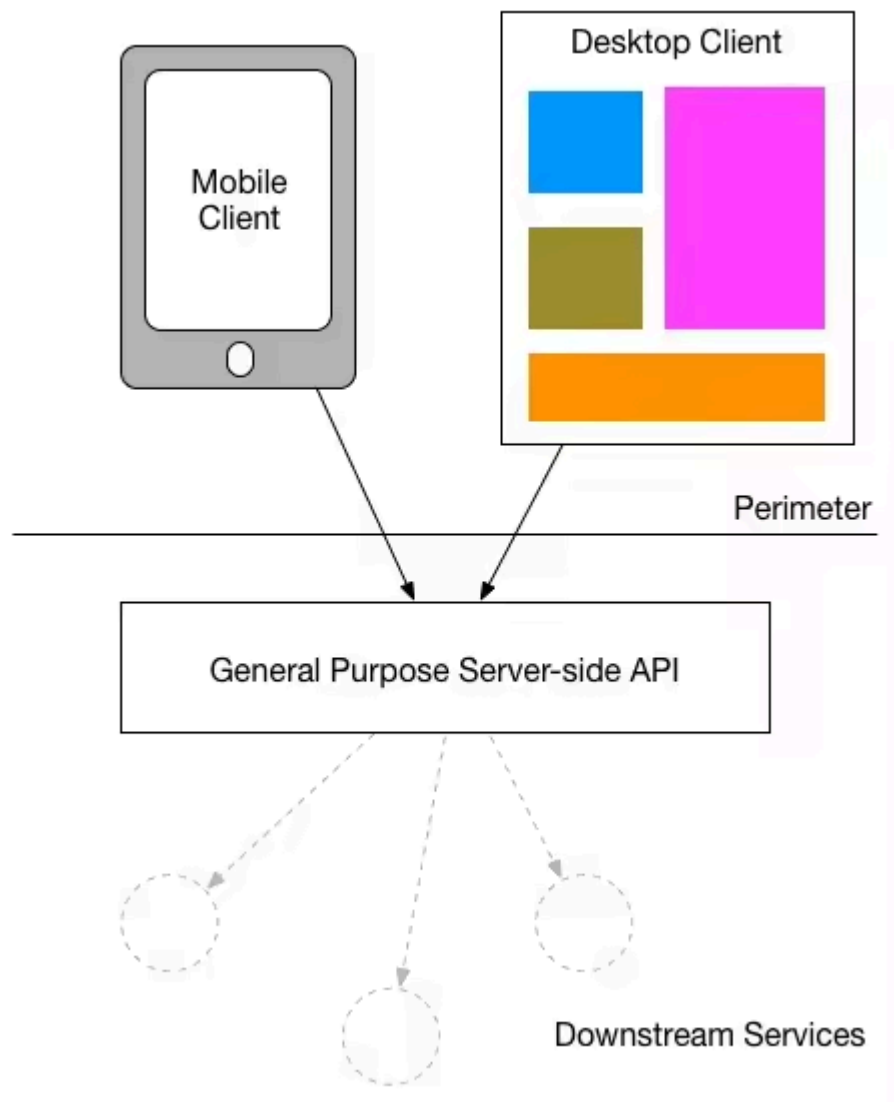


BFF调研

什么是BFF

BFF，即 Backend For Frontend（服务于前端的后端），起源于15年Sam Newman的一篇文章——[Pattern: Backends For Frontends](#)。

BFF，其实简单来说，就是服务端设计 API 时考虑前端的使用，在服务端完成全部业务逻辑的处理，所以也称为用户体验适配器。

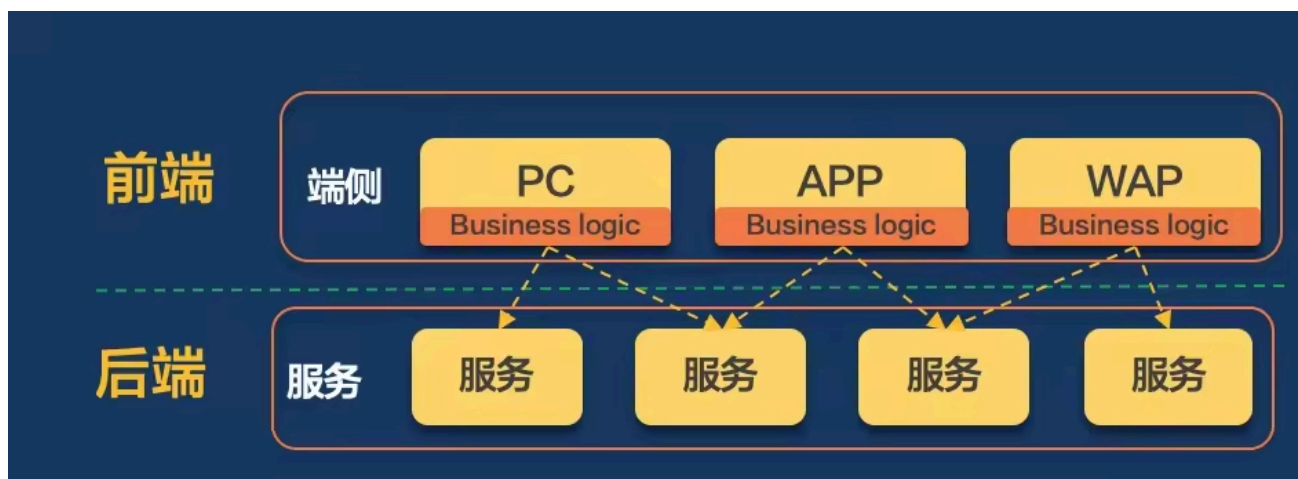


2 为什么需要BFF

服务端设计的API接口，到底是应该面向通用服务，还是要面向UI？

- 如果面向通用服务的话：那前端需要对接口进行二次加工，适配到UI层。这样，前端的展示组件中会耦合太多的业务逻辑，组件的通用性也会大大降低。

- 如果是面向UI的话：UI层是频繁变化的，后端需要为这些变化一一开发相应的服务，通常情况下，这些变化的服务复用性基本为零。后端的开发成本会大大增加。



这一时期内，后端和前端达成了某种意义上的一致：由于端侧请求的昂贵，后端负责了一部分的聚合逻辑，前端也维护了一部分的UI逻辑。这时的架构基本如下：



随着时代的发展，用户的接入形式也越来越多样。即使不同设备可能是实现相同的功能，但因为不同设备的特殊性，它们对服务端的API访问也各有其特点，需要区别处理。在这种情况下，后端不得不为每个端开发了对应的API，架构大致如下：



这种架构下，后端开发压力与日俱增，同时，这种情况下前后端各自负责了一部分UI逻辑，架构上也很不合理。

BFF理念：将内部复杂的微服务，适配成对各种不同用户体验（无线/Web/H5/第三方等）友好和统一的API。BFF相当于是前端和后端之间的一个中间层（适配层），作为前后端之间的桥梁，增加灵活性。BFF的提出，其实是开发人员在重新探索更合理的分层协作模式。

BFF理念提出后，**端用户体验层->网关层->BFF层->微服务层**，是现代微服务架构的典型分层方式，这个架构能够灵活应对业务需求的变化，是一种支持创新的演化式架构。



3 BFF中的三类角色

3.1 微服务提供者

微服务提供者的诉求就是对业务模型进行沉淀。

微服务本身是对业务的实现，但对业务模型的抽象主要在数据库的字段上面。这些字段的关联性和相关接口的代码组成了业务的模型。

业务模型可以通过表结构和它们之间的描述描绘出来，或是通过ORM来描述，但都不如GraphQL的Schema来的直接和直观，而且除了模型描述，这些Schema还有更多的好处，比如数据Mock。

难点：

微服务提供者写微服务的时候**遵循的标准和规范**，是否可以快速低成本地被应用到一些原本就存在的微服务上，降级方案是什么？这种标准和规范要同时适用于Java的HSF服务和Node.js提供的HSF服务。

3.2 网关提供者

网关提供者可以依赖**遵从相同标准和规范**而书写的微服务，对外提供**注册、管理和服务编排**工作。

服务的编排就是指可以像写GraphQL的Query语句那样，在Query中聚合所需要的服务，同时指定要输出的字段。这样既可以不用写代码就可以实现多服务的聚合，又可以实现定制化接口，达到“要什么，给什么”的效果。类似下面的Query:

∨ 复制代码

```
{
  gw_auth(accessToken: "77543e3b888a43eebe10832accc8c50000d50f00"),
  demo_getCreditCardNo(userId: @gw_auth.userId),
  demo_getCreditCardInfo(cardNo: @demo_getCreditCardNo, userId:
@gw_auth.userId)
  {
    name,
    workNo,
    validTerm,
    workNo
  }
}
```

难点：

提供业务模型视图，圈进业务需要的微服务，进行分治，生成业务的模型，同时提供出业务的Schema。这个Schema是前端Mock数据的基础。

一个业务只有一个Schema，在这个Schema的基础上衍生出众多的Query，而不是众多的接口。

3.3 前端调用者

前端的诉求就是接口可定制、可Mock，这样可以实现更加彻底的前后端分离。

难点：

更深层次地，前端可以通过标准化的数据查询DSL，书写标准化的语法（相当于适配层），这种语法屏蔽了网关提供者的Query语法，这样的好处就是，当网关提供者的Query语法变动后，只需要修改DSL，而不需要前端调用者修改每一个Query。

4 Artisan：基于GraphQL的BFF实践

4.1 核心实现原理

Artisan接收请求后，根据规则路由(PageSource)到某个页面(Page)，一个页面由多个组件（区块）构成，每个区块各自包含下游数据提供者(DataProvider，下文简称DPI)。

DPI是后端页面开发的原子单位，开发完DPI剩下的全是配置了。日常维护基本都是在组件和DPI的编排上，在组件维度上将不同服务做相应的适配给到前端。

8:49



支付宝商家服务



芬奇

升级为人气商家

收款(元)

今天 昨天

账户余额(元)

0.01

1.75

共1笔

查看明细

资金管理

今日新增提现免费额度



开店获客



报名消费券活动

支付宝出资 为你引流

立即报名



访问量

优惠券使用量

--

--

资金助手



天天算收益
灵活取用

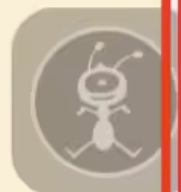
查看余额宝 >

更多服务

小店福利

5元红包待领取

- 小店进货补贴
- 每天来都可领1次





4.2 BFF适配层功能

4.2.1 聚合

- 多维度服务聚合：多个dpi数据聚合，如APP端常见的个人页面里，常常需要将用户的个人信息接口 和 用户的偏好商品接口合并。
- 单维度服务聚合：单个dpi数据内聚合，如商品的信息来自多个团队，基本信息、营销信息、DA离线统计信息、供应链信息等等，所有数据在研发过程中内聚到一个DPI。

4.2.2 数据按需使用

- 按需使用：不同的端消费的字段不一致，服务端只需定义通用服务，消费方各自取各自所需的字段，冗余字段不会出现在报文和网络IO中。
- 隐私字段：后端微服务里返回的隐私字段不能在前端接口直接输出。

4.2.3 结构调整

- 字段别名：如在前端组件里，商品标题的字段名为title，后端返回的字段名为name
- 将接口返回层次化结构拍平：如后端返回的结构是

```
{
  "company": {
    "name": "公司名称",
    "url": "公司链接"
```

复制代码

```
}
```

而组件侧需要的结构是



复制代码

```
{  
  "companyName": "公司名称",  
  "companyUrl": "公司链接"  
}
```

4.2.4 MOCK

MOCK功能是解决前端、后端、算法间的互相依赖、互相等待问题的重要利器。提供与访问真实线上环境一致的MOCK服务API是前后端同学一直追求的梦想。

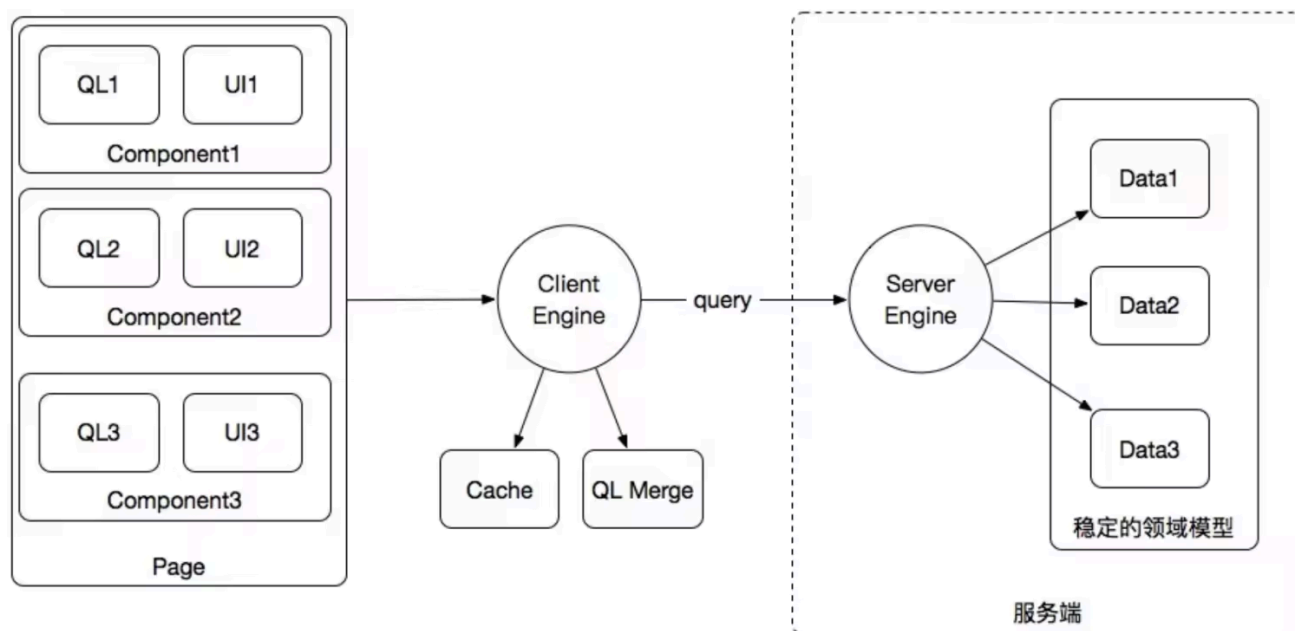
4.3 不能做什么

- 不涉及业务组件的前端实现
- 不适合有复杂UI联动交互和提交的页面

总的来说，Artisan适用于注重pv的页面场景。

表单类需求和企业后台管理页面需求，这类需求往往涉及页面多个部分之间复杂联动。同时，提交数据是业务组件内部逻辑，提交自定义接口，Artisan只负责页面的展示渲染。

4.4 数据源



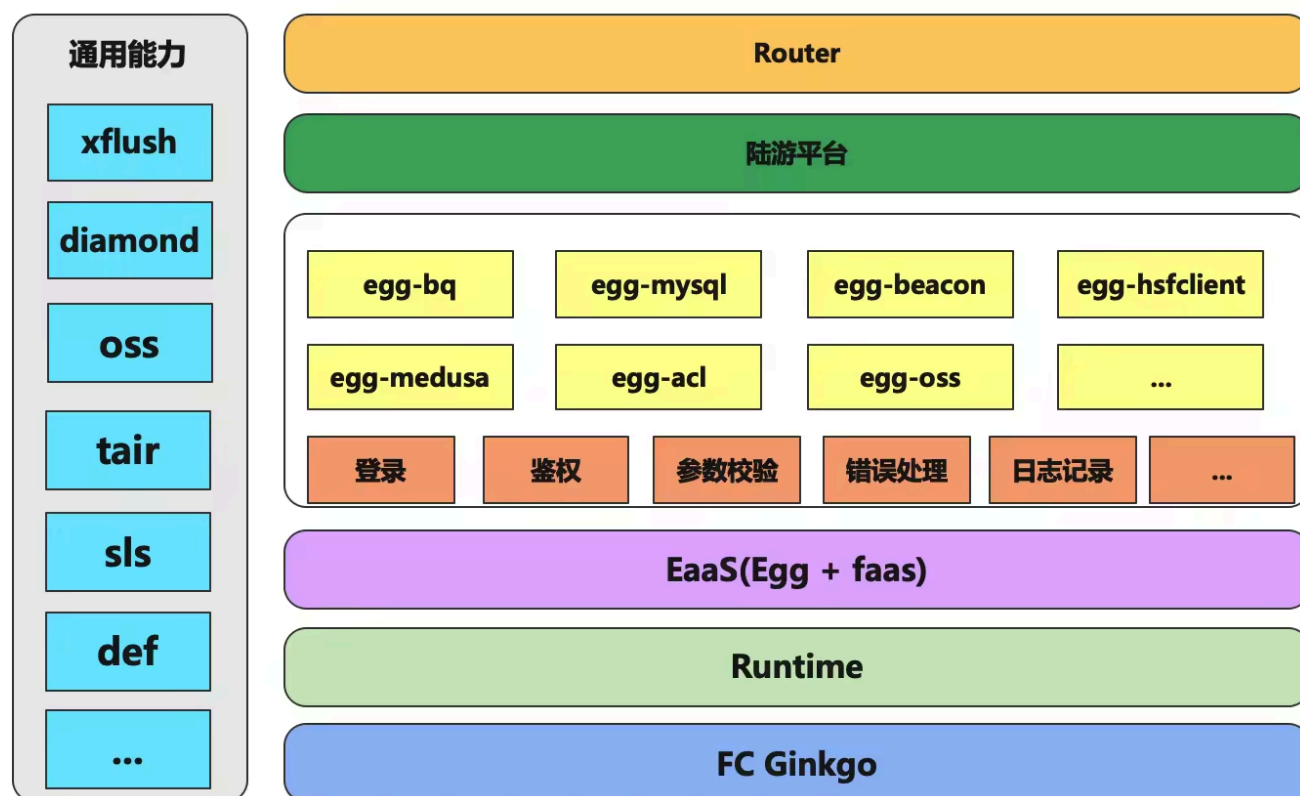
5 ICBU广告BFF落地实践

5.1 问题描述

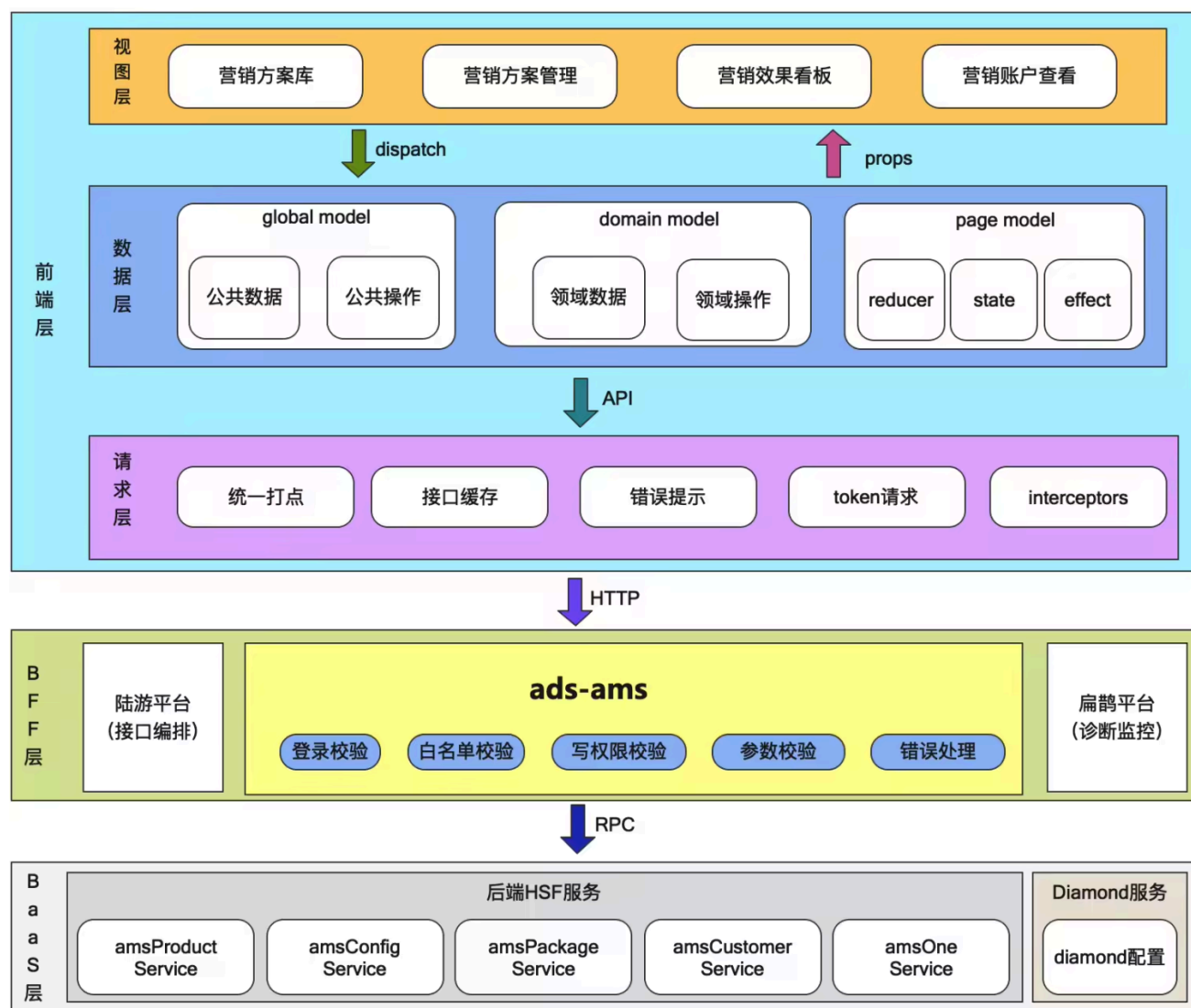
目前前后端的分层架构中，vm层耦合严重，开发协同效率较低，vm层放在后端web层的应用上，但又是主要由前端同学负责开发和编写，导致职责不清，管理混乱。比如以广告BP后台的ads-web应用为例，它负责了整个广告的web接入层，其中大部分是由后端java开发编写，vm模版由前端同学负责维护，常常导致以下问题：

- 这个应用非常庞大，管理了所有前端页面的vm，发布异常繁忙，发布堵车问题时有发生。
- 耦合了前端的vm模版，如果一旦部署或者挂掉，则前端开发直接中断，aone的部署又很慢，经常动则10分钟，导致前端联调过程中很痛苦。
- 前端想基于模版做个优化非常困难，因为和后端耦合在一起，需要改动和发布后端应用，由于安全生产、测试、发布以及关键应用的严格管控等等因素，往往前端去发布和改动后端应用会非常谨慎，所以前端不到万不得已都不会去改模版，这就导致前端失去了一个重要的优化手段。

5.2 基于Node的可视化逻辑编排与代码热更新平台



5.3 全域通BFF落地实践



5.4 收益

5.4.1 研发提效

•前端提效:

- 与现有java webx相比，落地BFF架构之后，前端整体估计**提效10%以上**；
- 落地BFF架构之后，前端会增加BFF接口的开发成本，但是前期定义好的话，在陆游平台上开发BFF接口非常简单清晰，**部署秒级生效**，开发BFF接口的时间占整体时间10%左右；
- 前端页面部分的接口逻辑会非常清晰，接口交互和适配的逻辑都放到BFF上；
- 加入BFF之后，后端的接口字段变化，前端的兜底逻辑等都不会传导到前端层，这会大大降低联调时需求变动或者后端接口字段变动导致的前端修改成本。

•后端提效:

- 落地BFF架构之后，前端接管了web层，后端不再需要写web层接口。

5.4.2 业务价值

- 快速响应：

- node的引入，不需要后端统一封装成HTTP接口来提供给前端，这样可以大大节省中间的人力成本，让业务可以更加快速的跑起来，快速迭代快速试错。

- 拓展边界：

- 跨团队协作，优先级和沟通效率得到保障；

- 通过node的引入，更加低成本和高效的支持业务成长。